

DynBranch: Speculative Subgraph Reuse for Dynamic Agentic LLM Serving

Junyi Shen

National University of Singapore
Singapore
j1shen@comp.nus.edu.sg

Zhengyuan Su

National University of Singapore
Singapore
suzhengy@comp.nus.edu.sg

Noppanat Wadlom

National University of Singapore
Singapore
noppanat@comp.nus.edu.sg

Yao Lu

National University of Singapore
Singapore
luyao@comp.nus.edu.sg

Abstract

Agentic LLM workflows decide their execution paths at run-time. Downstream computation may be predictable, or may have run before, yet it cannot begin until the model or the user resolves the branch. We call this serialization the *branch-resolution barrier*. Caching alone does not hide it: the key that identifies a reusable result is not known until then.

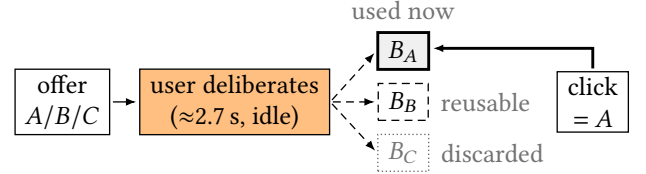
In this paper, we propose DynBranch, which makes an unresolved branch addressable before it resolves. Its stable *coordinate* lets candidate subgraphs run during resolution and completed subgraph results be reused across later requests. A two-level controller admits this work when its expected benefit exceeds the load price. DynBranch sits at the model-API boundary and requires no changes to agent harnesses or model execution engines.

Across four agentic workloads with Qwen3-32B on 4× H200 GPUs, DynBranch reduces mean latency by up to 32% over each workload’s strongest prior system and by 46–66% against a no-reuse floor, while preserving workflow results. The benefit persists across backbone families and on a commodity Qwen3-8B/RTX 4090 deployment.

1 Introduction

LLM applications are moving from workflows whose model and tool calls are fixed before execution [38, 52] to agentic ones that decide each next step as they run, by the model or by the user [101, 104, 108]. Measured agent workloads show tens to hundreds of model and tool calls per task [55, 120], so these decision windows repeatedly enter the critical path. Downstream work waits on a choice the model has yet to decode: the handler a request needs, the sub-task a sub-agent runs, the entity a retrieval fetches. Each is typically followed by model inference. For example, a ReAct agent answering multi-hop questions decodes the entity it will search next [108], and coding agents such as Claude Code and Codex show both: they decode the next tool call, and at interactive decision points they wait for the user to pick from a short list of suggested actions [5, 63].

(a) human-wait window — indep. top-K



(b) model-decode window — exclusive

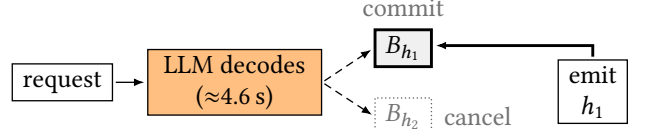


Figure 1. Predicted subgraphs fill during the shaded resolution window; each candidate’s status is printed beside it. (a) independent top-K. (b) exclusive route.

We call this serialization the *branch-resolution barrier*: downstream computation that may be predictable, or may have run before, waits for the model output or user choice that determines the branch. In measured coding, research, and scientific agents, 45–57% of end-to-end latency is downstream time serialized after model output [88]. This creates two opportunities. Starting likely downstream computation before resolution can hide part of its latency on the first occurrence (Figure 1). Reusing a validated result can avoid that computation on later occurrences.

Existing systems provide result caching [9, 94] and agent-level speculation [28, 34, 109] as separate mechanisms. Caches retain completed responses and operator results, while speculative agents consume or discard predicted work within the current request. Prefix caches reuse KV state [116], and dataflow schedulers overlap work once its inputs are known [38, 52, 89, 102]. These systems do not jointly support reusing a subgraph result across later requests, starting a branch before it resolves, and pricing that work against serving load. Each is hard in a different way. *C1, shared-subgraph reuse*: the unit that recurs is a sub-DAG rather than a whole request, and moving the boundary there must not weaken exact matching or freshness. *C2, early*

fill during resolution: the name that authorizes reuse does not exist until the branch resolves, yet the work must start before it does. *C3, load-priced admission*: speculative work draws on the same slots and KV as the request it accelerates, so what helps at low load hurts at high load.

In this paper, we propose DynBranch, which makes an unresolved branch addressable before it resolves (§3). For C1, it records a completed branch subgraph together with the input that produced it and the external reads it made. A later request consumes the entry only when its input matches exactly and those reads are still fresh; a write to any of them invalidates it. For C2, the workflow template provides a *coordinate*: the identity of a decision point in the template, such as the router stage of a routing workflow, known before the branch resolves in our target workloads (§5.6). Predicted candidate subgraphs run as *fills* during the resolution window; completed results can also serve later requests. Fills remain invisible until branch selection and an exact, fresh *canonical* (non-speculative) demand authorize their use, so wrong predictions cannot change workflow results. For C3, a two-level controller decides whether to generate candidates and whether to execute each fill, admitting either when expected benefit exceeds the load price over the fill’s execution horizon. That price reads the engine’s current decode and KV occupancy, so it rises as speculative and canonical work accumulate.

We implement DynBranch at the model-API boundary between agent harnesses and unmodified SGLang or vLLM engines [39, 116], an interface that covers both framework-driven workflows [25, 41, 101] and text-based coding agents [5, 21, 63]. With Qwen3-32B on 4× H200 GPUs, DynBranch reduces mean latency by up to 32% over each workload’s strongest prior system on the same GPUs and by 46–66% against a no-reuse floor, while preserving workflow results. With reuse disabled, early fill alone reduces mean latency by 8.5–34.4% on model-driven paths and user-turn latency by 46.6–47.7% on *HCI* (§5.4). The benefit persists across backbone families and on a commodity Qwen3-8B/RTX 4090 deployment.

We make three contributions:

- We identify the branch-resolution barrier and quantify its cost on production traces and four agentic workloads (§2).
- We propose a branch-state abstraction connecting speculative execution with cross-request subgraph reuse, promoted on exact match and invalidated by writes (§3.2–§3.3).
- We design a two-level online controller that prices candidate generation and fill execution against benefit and load, learning from delayed feedback (§3.4, §5.4–§5.5).

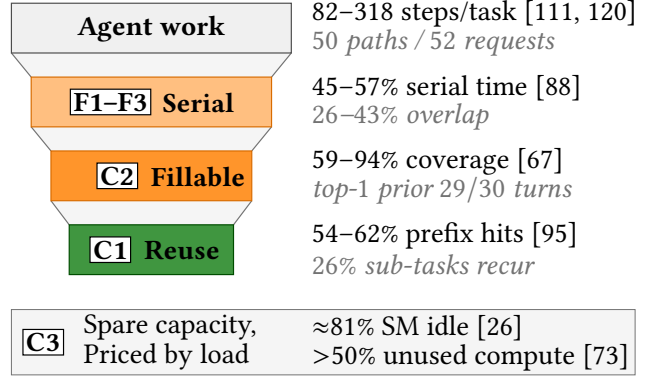


Figure 2. Opportunities for early execution (C2) and cross-request reuse (C1), subject to available serving capacity (C3). Upright figures are from prior work; italic figures are our own measurements with speculation disabled.

2 Background and Motivation

An agentic workflow interleaves model calls, tool calls, and user turns, and at many steps the next one depends on the current step’s output: the model names the tool or sub-task to run, or the user picks among offered actions. We call each model or tool call, executed on its rendered input (the prompt or tool arguments after upstream outputs are substituted), a *stage*, and the stages downstream of a decision point its *subgraph*. Figure 2 narrows from all agent work to the part serialized behind such decisions, then to the part that can start before a decision resolves (C2) and the part a later request can reuse (C1), all drawn from capacity that canonical requests also need (C3).

C1: shared-subgraph reuse. GPTCache and agentic plan caching match whole responses or plans [9, 74, 112], while SGLang, Pythia, PBKV, and KVFlow retain prefix-KV state but still execute the decode [64, 110, 115, 116]. Both anchor at the request boundary, at the whole request or its first token, so neither can name a sub-DAG that recurs inside requests differing elsewhere. Its retrieval and tool results also depend on external reads, so exact matching is not enough: writes to recorded dependencies must invalidate them (§3.2).

C2: early fill during resolution. KVFlow, Pythia, and PBKV prefetch or retain prefix state for a declared or predicted next stage [64, 110, 115], but the stage itself still runs only after resolution. Speculative Actions executes a predicted API call early, SPAgent a search action [34, 109], DSP a chain of planning steps [28], and Speculative Interaction Agents dependent tool calls alongside model reasoning [32]. PASTE, the closest, isolates a predicted tool result until a matching authoritative invocation commits it [88]. All five, however, can use a speculative result only in the request that produced it: the result is checked against that request’s next step and used or discarded there, so even a correct result is lost when the request ends. Early fill therefore needs results

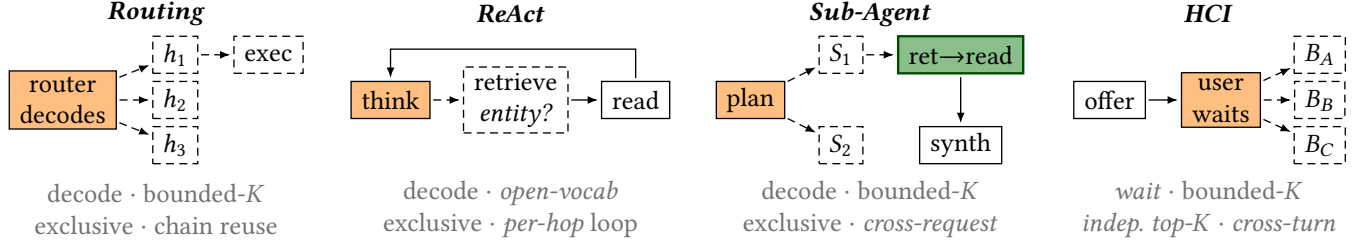


Figure 3. Four representative workloads. Shaded boxes are resolution windows; dashed boxes are candidate work; the heavy-outlined box is a subgraph that recurs across requests. Italic labels differ from *Routing*.

that stay isolated until a demand selects them, yet outlive the request that produced them, which requires naming them before the branch resolves (§3.3).

C3: load-priced admission. Fills run on the same GPUs as the requests they accelerate, so each unused fill delays canonical traffic (Figure 5b), more so at high load [4]. PASTE, SPAgent, TurboSpec, and Nightjar limit speculative work under load [34, 48, 54, 88], but each scores a unit once, before it runs. A branch incurs two costs, running the predictor and then executing the admitted candidates, and whether either paid off is known only when the branch resolves, often seconds later. Admission therefore needs a cost that tracks current load and a benefit estimate corrected by these delayed outcomes (§3.4).

What a workload must supply. First, the work needs an identity independent of runtime content, since conventional memoization names a result only once its inputs are known [9, 112]; a workflow template supplies one by naming each decision point, though the model still produces the selection or tool arguments (§5.6). Second, the stage must be safe to run twice: no external side effects, same result for the same input and dependency versions. Greedy inference with batch-invariant kernels [31] over an immutable model and read-only retrieval over a versioned snapshot qualify; stages that sample or mutate external state run only after resolution (§3.3). §2.1 describes four workloads that meet both, and §2.2 measures what each challenge is worth on them.

2.1 Dynamic Agentic Workloads

Agentic workflows determine downstream steps at runtime [5, 101, 108], and the decisions are frequent and diverse [56, 120]: agent sessions and tasks run 82–318 model steps [111, 120], and we measure 50 distinct paths in 52 requests, so every run opens tens to hundreds of resolution windows in which to speculate.

A workload taxonomy. Dynamic branches vary along four dimensions that set the benefit of early execution and the life of its results: the *resolution window* (model decode or human wait), the *candidate space* (bounded or open vocabulary), the *promotion semantics* (one selected winner or multiple valid

results), and the *reuse horizon* (within a chain, across turns, or across requests). Figure 3 places four workloads on them. *Routing* is a tool-use workflow over BFCL function calls [66], *ReAct* [108] a multi-hop question-answering agent [90, 106], *Sub-Agent* [101] an orchestrator that dispatches tasks to sub-agents, and *HCI* an assistant that offers actions and waits for the user to pick one [14, 22].

2.2 The Branch-Resolution Barrier

Most task latency is serialized. In coding, research, and scientific agents, downstream execution takes 45–57% of task latency on the serial model–tool path [88]. At branch k , work of duration T_k^{exec} waits for a resolution window T_k^{gen} , which for *HCI* is the user’s sampled think time [14, 22]. Early execution can overlap at most $\min(T_k^{\text{gen}}, T_k^{\text{exec}})$; summed along the critical path and normalized by wall-clock, this gives the workload’s *overlap share*, the most C2 can hide.

Figure 4 shows three observations. *F1, substantial latency can be overlapped:* the decode-window workloads can overlap 26–43% of wall-clock, and *HCI*’s post-click execution fits entirely within the modeled think window. *F2, the window cannot easily be shortened:* on *Routing*, the resolution decode and the downstream prefill carry similar token volumes (270 vs. 263 at p50), yet only the former is emitted autoregressively [3, 39], and bounding that decode can change the answer [42, 60]. *F3, window duration limits the overlap:* at p50, outside *HCI*, the resolution window is 0.36–0.77× as long as the downstream execution.

Predictability determines coverage cost. Early execution saves time only if the admitted set contains the branch eventually selected, and predictability differs by class. A bounded space is enumerable from the template, and a skewed mix lets a few candidates cover most traffic: bounded API selection reaches 91–94% with an oracle retriever and 59–84% with a deployed one [67]; Figure 2 spans both. A user’s next action follows a frequency prior over those offered before [20, 23]; in *HCI* the top candidate matches the selection in 29 of 30 turns. An open vocabulary has neither [81], so candidates come from context already read: our *ReAct* predictor ranks entities named in the previous hop’s passage, placing the next in its top three 61% of the time. More candidates improve coverage [36] at proportionate cost.

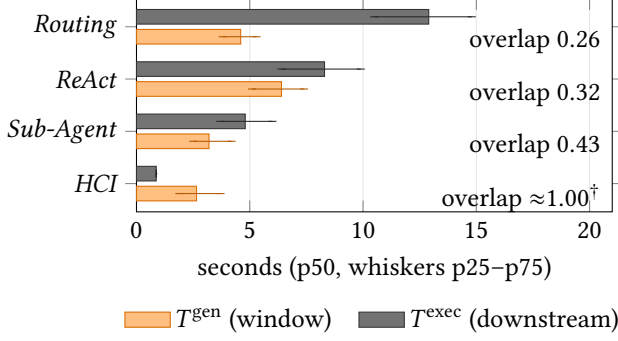


Figure 4. Resolution windows and downstream execution times; bars show p50, whiskers p25–p75. † Normalized by post-click execution, not wall-clock.

Recurrence concentrates in content sub-tasks. What a fill cannot hide, reuse removes outright when the same work is asked for twice. With speculation disabled, 12% of *ReAct* retrievals in a run and 26% of *Sub-Agent* sub-tasks across runs have byte-identical inputs, and recurrence increases as more requests arrive (Figure 5a): repeated MuSiQue [90] sub-topic mentions rise from 19% at 100 questions to 61% at 5,000. It appears outside our workloads too: our analysis of released SWE-agent and SWE-rebench trajectories [7, 104] finds 42–45% of commands recurring verbatim within a repository. A production prefix-cache study reports 54–62% ideal hits [95]; that unit requires a shared prefix, while the retrievals and sub-tasks above recur in requests that differ elsewhere: the unit C1 must name.

Speculation’s cost depends on load. A fill’s engine time pays off only if its result is used. Serving leaves capacity for it [65, 99]: over a day, GPUs leave $\approx 81\%$ of streaming-multiprocessor (SM) time idle [26], decode leaves $>50\%$ of compute unused [73], and human thinking is 64% of coding-session wall-clock with idle gaps capped at an hour [120]. Fills still compete with canonical requests for decode slots and memory bandwidth; Figure 5b measures p50 latency for a fixed-shape 32B request as fills grow in number and context. At a fixed context length, 1–3 co-resident fills raise it by 4–6% and 48 by 53%; holding the count at 24 and growing each context from none to 4,500 words raises it from 11% to 145%, as the fills grow from 3% to 24% of the KV pool. Resident KV rises with both, which is why C3’s admission reads it as the measure of speculation in flight (§3.4).

3 DynBranch Design

3.1 System Overview

DynBranch is a serving layer between agentic harnesses and model-serving engines, built from one interface and three parts. The *candidate interface* takes a branch coordinate and returns a ranked candidate list; which predictor sits behind it is a deployment choice (§3.3). A two-level *gate* prices whether to open a branch, and which of its candidates to fill, against

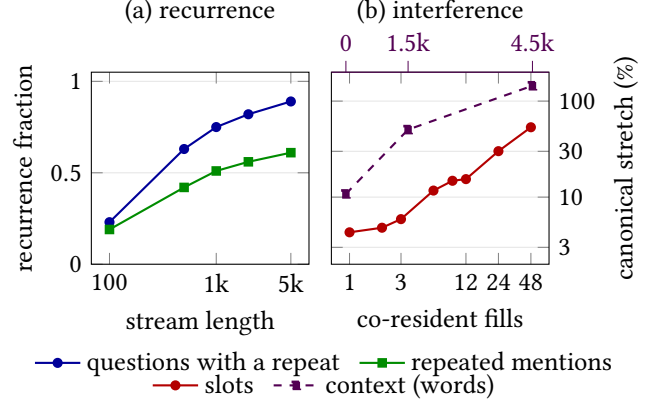


Figure 5. (a) Share of MuSiQue questions that repeat an earlier sub-topic, and of sub-topic mentions that are repeats, as the stream grows. (b) Slowdown of a canonical request as fills grow in number and context.

current load. The *Completed-Subgraph Library* (CSL) holds finished subgraph results for later reuse, and a reverse index from the external records they read invalidates them. A per-branch record follows each open branch until it resolves and returns what it cost and what it saved to the gate.

Life of a request. Figure 6 follows a model-decided branch: its resolution window lets predicted downstream work run before the next stage is named. The predictor uses the branch *coordinate*, the stable identity of that decision in the workflow template, to propose downstream candidates (①). The gate prices each candidate and admits those whose expected benefit exceeds its load price (②); the admitted fills then run on the same engines as canonical traffic (③), concurrently with the branch-resolution decode.

A completed fill remains an invisible *shadow*, a speculative result that serving paths cannot read (④). Branch resolution eliminates losers but does not authorize the survivor: only a subsequent canonical demand with the same exact reuse name can promote and consume it (⑤). This separation takes the serialized work off the critical path without exposing unchosen work. The confirmed entry then serves later exact matches (⑥) until a dependency write invalidates it (⑦).

DynBranch treats reuse, overlap, and execution as separate decisions. *C1: what may be reused?* A subgraph, identified by its exact realized input and fresh dependencies (§3.2). *C2: how can work run before resolution?* A stable coordinate starts fills during the window; a matching canonical demand authorizes use (§3.3). *C3: when should that work run?* When its expected latency benefit exceeds its load-priced resource cost (§3.4).

3.2 Shared-Subgraph Reuse (C1)

C1 must capture reuse that exists below the whole-request boundary or across requests that differ elsewhere, without serving a hit it cannot replay. Replay safety asks more than

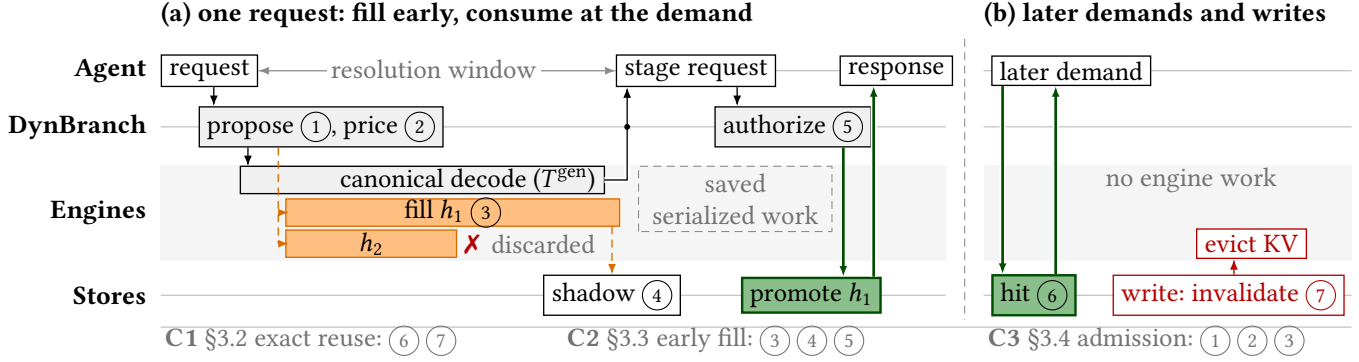


Figure 6. Lifecycle of a speculative branch result. Lanes are components, time runs left to right; $\times$ marks the discarded loser.

a semantic cache’s approximate match [74]: every result-determining input must match, every external dependency must remain fresh, and a multi-node result must come from one execution, not fragments of several.

Subgraph reuse names. Exact reuse follows the dependency closure captured by a rendered input, not a node’s graph position. A whole-response key needs the whole workflow to repeat [9] and a prefix-region key an unchanged prefix [64, 116]. DynBranch instead makes the unit a self-contained subgraph, depending only on the upstream values entering its own rendered inputs, so it stays valid beyond an unrelated change. It records that subgraph with the realized input, output, external read-set, and provenance of one execution. Its *reuse name* combines a subgraph fingerprint with a hash of the realized input, covering the request fields that determine the output. C2 addresses work before resolution through the branch coordinate, which identifies an unresolved decision and therefore cannot authorize reuse.

Freshness is maintained by a reverse index from external records to cached entries. A write invalidates direct dependents and any cached or engine-resident state that loses a valid producer; incomplete provenance conservatively extends the cascade. Reads synchronize with invalidation, so concurrent or uncertain cases fall back to a miss. Writes may reduce reuse but cannot expose stale state; the cascade follows view-maintenance and lineage work [1, 11, 27]. Together, these checks make a hit replay one recorded execution [61].

Benefit of reuse. Let T_j^{sub} be the execution the subgraph rooted at decision point j adds beyond its nested subgraphs, and P_j^{hit} the probability that a later request is served it from cache while its recorded reads stay fresh. In expectation, reuse removes

$$\mathbb{E}[\Delta T_{\text{reuse}}] = \sum_j P_j^{\text{hit}} T_j^{\text{sub}} \quad (1)$$

from that request’s critical path. The boundary sets P^{hit} : under one realized change (Figure 7), the whole-response, prefix-region, and subgraph boundaries serve 0, 2, and 4 of six taken nodes at the same exactness and freshness. A cache

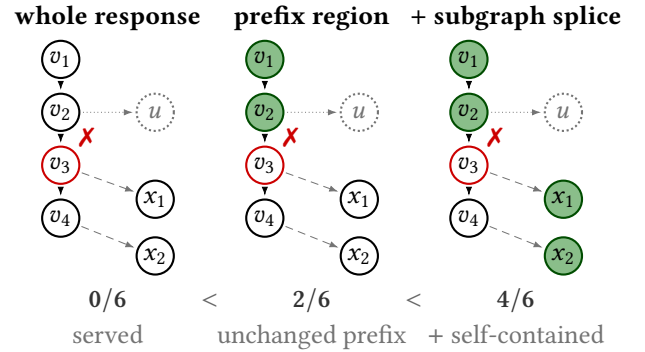


Figure 7. One realized change (red $\times$) under three exact serving boundaries. Green: served from cache; dashed arrows reach control-only descendants; dotted u is an untaken branch. Footers: nodes served of six taken.

that does not model what the stage computes cannot match more often than this and stay exact, so promotion refuses nothing a safe rule could admit (Corollary A.4). C3 values only the execution Equation (1) leaves.

On a canonical demand, C1 derives the reuse name and probes the CSL’s confirmed entries under the freshness guard. A hit returns the recorded result; a miss executes normally and may refill the CSL, which is bounded and evicts by recency, frequency, and cost per byte.

3.3 Early Fill During Resolution (C2)

C2 must avoid two failures. The first is serial: downstream work that could have run inside the resolution window waits for it instead, and the obvious repair (publishing a result as soon as it completes) serves work no demand selected [82]. The second is cross-request: speculative decoding, speculative agents, and speculative tool systems bind validation and consumption to the request that authorized the work [28, 32, 43, 88, 109], so a result that survives its branch still dies with its request. C2 therefore decouples computation, selection, and authorization.

Where candidates come from. Any predictor can sit behind the interface: a next-stage model over the coordinate’s

history, a retrieval ranker, or a small draft LM sampled N times and ranked by how often it names a candidate (§5.3). Only its ordering is used, since its scores are uncalibrated; a per-rank consumption rate, learned online, replaces them.

Speculative results and promotion. C2 speculates only on *speculable* stages, those the template declares safe to run twice (§2); unannotated, sampling, or externally mutating stages run only after resolution. Admission turns a candidate into a *fill*. Once the fill’s input renders, C1 derives the reuse name, and completion records that name, the result, and its read-set as a persistent shadow, visible to the gate’s per-branch record but unreadable by serving paths.

A canonical demand resolves this lifecycle in two steps. First, selection settles the branch. For an exclusive branch, the resolved choice retains the matching candidate and aborts the losers’ in-flight fills in the engine, releasing their slots and KV and discarding their results; independent offers remain eligible, their shadows unreadable until an exact demand authorizes them or they expire. An ambiguous branch or choice promotes nothing. Second, authorization applies C1’s guards: the canonical reuse name must match the shadow’s, and its recorded read-set must remain fresh. Only then is the shadow atomically promoted and consumed: selection eliminates work, content and freshness serve it.

If the matching fill is still running, the canonical demand waits a bounded interval and otherwise executes normally. Multi-stage fills are promoted stage by stage as canonical demands pass the same guards.

Speculative–canonical equivalence. Let $F(q, \theta, D)$ be a speculable stage run with rendered request q , model and execution parameters θ , and dependency versions D , and write s and c for its speculative and canonical executions. Promotion requires a reuse-name match under the freshness guard. The name covers the stage fingerprint, the complete rendered input, and both parameter sets, so a match gives $(q_s, \theta_s) = (q_c, \theta_c)$, and the guard gives $D_s = D_c$. Determinism then yields

$$y_s = F(q_s, \theta_s, D_s) = F(q_c, \theta_c, D_c) = y_c,$$

and a promoted stage i renders to stage $i + 1$ the input its canonical execution would have rendered, re-establishing the hypothesis along a multi-stage fill. Two properties follow. *Value equivalence*: a promoted subgraph returns exactly what canonical execution would. *Non-interference*: unpromoted work is never observed—no external effect, no readable state—and exclusive losers do not outlive their branch. A failed precondition is a miss, so speculation changes only latency.

Benefit of early fill. At branch k , let T_k^{gen} be the generation window that resolves the branch, T_k^{exec} the selected downstream work, and P_k^{cov} the probability that the admitted set contains the selected candidate. Ignoring contention, early fill at branch k hides at most $\min(T_k^{\text{gen}}, T_k^{\text{exec}})$ if the admitted set covers the selection and nothing otherwise (Figure 4).

With the durations fixed, in expectation over coverage:

$$\mathbb{E}[\Delta T_{\text{gross}}] \leq \sum_k P_k^{\text{cov}} \min(T_k^{\text{gen}}, T_k^{\text{exec}}). \quad (2)$$

Equation (2) is a gross bound (Lemma A.6): C3 prices contention explicitly [4, 86, 93], and C2 admits fills only while a positive overlap window remains. The opportunity grows with branch predictability, downstream cost, and window length. For a user-wait window, the same expression substitutes think time for generation time and measures the reduction in post-click latency [12, 22].

3.4 Load-Priced Admission (C3)

DynBranch admits speculative work in two steps (Algorithm 1). For each branch, the *generation gate* (Action 0) decides whether running the predictor and expanding candidates is worth its fixed cost. If it opens, the *fill gate* (Action 1) admits a candidate only if the expected time it saves exceeds the expected delay it imposes on canonical requests at current load, re-pricing after each admission. The two are not one decision: generation itself costs time that no fill threshold can recover, and a cheap generation step can still reveal fills that are individually too costly.

Predicting more actions raises the chance of covering the realized branch, but also creates work that may never be used [34, 58, 105, 109]. Admission must therefore protect canonical demand from those fills. Paying for an alternative before knowing whether it is needed is classical optimal search [97]; here the alternative is produced, not fetched, so its price moves with contention.

Causal load price. At time t , C3 observes a load view V_t comprising normalized fill occupancy o_{fill} , reservation-budget occupancy o_{KV} , and canonical-demand pressure ρ_t . Admission is the only point at which contention can be refused: a running fill holds its slots until it completes or its branch cancels it, so an instantaneous idle reading would underprice a multi-second fill. C3 prices against the causal envelope [84]

$$\bar{\rho}_H(t) = \max\left\{\rho_t, \frac{1}{H} \int_{t-H}^t \rho_\tau d\tau\right\}. \quad (3)$$

Here H matches the window used to estimate speculative co-residency. Rising load takes effect immediately; falling load relaxes the price gradually over a full execution window. A periodic probe updates this history even while the generation gate is closed.

Candidate s carries footprint $\hat{\mathbf{R}}_s = (\hat{R}_{\text{slot},s}, \hat{R}_{\text{KV},s})$, covering decode time and KV residency weighted by its budget share. Monotone price curves $\boldsymbol{\beta}(V_H) = (\beta_{\text{slot}}, \beta_{\text{KV}})$, in milliseconds of canonical delay per unit of each resource, convert the footprint into a delay cost, $C_s(V_H) = \boldsymbol{\beta}(V_H)^\top \hat{\mathbf{R}}_s$, where V_H is the load view V_t with $\bar{\rho}_H$ in place of ρ_t . The slot price scales with $\bar{\rho}_H$ and the KV price with budget occupancy; both are initialized from Figure 5b and steepen near capacity. Measured

prices govern value; fill-count and atomic KV reservations remain hard feasibility constraints.

Generation gate (Action 0). A branch’s payoff is known only after it settles, so C3 learns one full-path return per branch entry:

$$\hat{y}_b = \sum_{s \in \text{consume}(b)} \hat{G}_s^\Delta - T_{\text{setup},b} - \hat{K}_b - \hat{W}_b. \quad (4)$$

Here $\text{consume}(b)$ is the set of fills branch b consumed and $\hat{G}_s^\Delta$ is the execution a consumed fill saves after C1 lookup, so the first term credits only the residual C1 left. The remaining terms charge measured prediction and expansion time $T_{\text{setup},b}$, the cost $\hat{K}_b$ of the fills this branch enabled, and $\hat{W}_b$ for expired or uncovered waits. Using one aggregate target is essential: summing separate optimism bonuses over mutually exclusive candidates would overvalue a branch.

A branch class c groups the branches that share a coordinate, so repeat visits to one decision point pool their statistics. For each class, context x summarizes horizon pressure, whether a low reading is transient, recent fill consumption, and queue backlog. A contextual ridge model maintains $A_c = I + \sum_{b \in c} x_b x_b^\top$ and $q_c = w_{0,c} + \sum_{b \in c} x_b \hat{y}_b$ from a class prior $w_{0,c}$. Following linear UCB [46], the gate opens when

$$\hat{U}_0(c, x) = x^\top A_c^{-1} q_c + \hat{\sigma}_c \sqrt{x^\top A_c^{-1} x} > 0, \quad (5)$$

where $\hat{\sigma}_c$ scales exploration by the observed target RMS. C3 allows n_0 cold trials per class, counted by n_c , before applying it, and registers each trial in the branch’s record before expansion, so feedback arriving during setup is not lost (App. A, generation gate).

A predictor too expensive to run inline is time-boxed off the dispatch path and yields an empty set at its deadline, which bounds $T_{\text{setup},b}$. The time it spent is charged either way, so a branch class whose predictor repeatedly pays setup and returns nothing consumable drives $\hat{y}_b$ negative and closes its own gate.

Fill gate (Action 1). For candidate s , the optimistic probability of useful service factors as $\tilde{p}_s = \tilde{\pi}_s^{\text{sel}} \pi_s^{\text{use}}$: a rank-conditioned Beta upper quantile [37] times the posterior mean of timely usability conditional on selection. It clears the priced rule when

$$U_s(V_H) = \tilde{p}_s \hat{G}_s^\Delta - (1 - \tilde{p}_s) C_s(V_H) > 0. \quad (6)$$

Both terms are measured in milliseconds. If $\hat{G}_s^\Delta + C_s(V_H) > 0$, the rule is equivalent to

$$\tilde{p}_s > p^*(s, V_H) = \frac{C_s(V_H)}{\hat{G}_s^\Delta + C_s(V_H)}. \quad (7)$$

The threshold is exact at the current projected load, so the same candidate can pass under headroom and fail under contention. On the exploitation path, Action 1 admits the highest-utility candidate that passes feasibility and atomic reservation, projects its pressure, and reprices the rest [68]. It stops when no remaining candidate has positive utility

Algorithm 1 Load-priced admission with delayed feedback.

Require: load history, posteriors, branch-class models

Action 0: generation gate for branch b

- 1: $c \leftarrow \text{BRANCHCLASS}(b); (x, V_H) \leftarrow \text{HORIZONCONTEXT}$
- 2: **if** $n_c < n_0 \leq n_c + n_c^{\text{pending}}$ **then return** $\triangleright$ cold trials in flight
- 3: **if** $n_c \geq n_0 \wedge \hat{U}_0(c, x) \leq 0 \wedge \neg \text{AUDIT}(b, 0)$ **then return**
- 4: $\tau \leftarrow \text{BEGINTRIAL}(b, c, x)$
- 5: $\mathcal{A} \leftarrow \text{ELIGIBLECANDIDATES}(b); \mathcal{S} \leftarrow \emptyset; V' \leftarrow V_H$

Action 1: fill gate at projected load

- 6: **while** $\mathcal{A} \neq \emptyset$ **do**
 - 7: $s \leftarrow \arg \max_{a \in \mathcal{A}} U_a(V'); \mathcal{A} \leftarrow \mathcal{A} \setminus \{s\}; \text{probe} \leftarrow 0$
 - 8: **if** $U_s(V') \leq 0$ **then**
 - 9: **if** $\neg \text{AUDIT}(b, 1)$ **then break; probe** $\leftarrow 1$
 - 10: **end if**
 - 11: **if** $\text{HARDFEASIBLE}(V', s) \wedge \text{RESERVE}(b, s)$ **then**
 - 12: record $(1 - \tilde{p}_s) C_s(V')$ and the producer identity
 - 13: $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}; V' \leftarrow \text{PROJECTLOAD}(V', s)$
 - 14: **end if**
 - 15: **if** $\text{probe} = 1$ **then break**
 - 16: **end while**
 - 17: $\text{SUBMITFILLS}(b, \mathcal{S}); \text{CHARGESETUP}(\tau, T_{\text{setup},b})$
 - Feedback: event e for trial τ**
 - 18: update selection, usability, gain, cost from e
 - 19: $\text{REVISETRIAL}(\tau, \Delta_e \hat{y}_b); \text{RELEASESETTLEDRESOURCES}(e)$
-

(Proposition A.8). Seeded audits occasionally admit refused work, so closed gates keep learning [91].

When the branch resolves, C3 updates selection for every predicted candidate, admitted or not; it updates usability only for admitted, selected fills, so one that misses its deadline for use after resolution lowers usability without becoming a prediction error. Delayed events revise the originating branch’s target rather than adding samples. Credit requires the consumed entry to name the speculative producer recorded at admission (§5.4). Pricing classes separate single- and multi-stage fills, so a chain’s cost is compared with its full saving.

The three contracts now form one lifecycle: C3 admits fills, C2 runs them during resolution while hiding their results, and exact, fresh C1 demands promote and reuse them. A C1 hit then suppresses the execution it answers and the speculation below it, unless an external wait follows the stage.

4 Implementation

Agentic harness integration. Framework-driven harnesses such as AutoGen [101], LangGraph [41], and AgentScope [25] attach to DynBranch through an HTTP interceptor that tags each model request with `workflow_type_id`, `workflow_id`, and `agent_id`. Text-based coding-agent harnesses such as Claude Code [5], Codex [63], and DeepSeek Harness [21] normalize into the same arrival representation from their own wire protocols, so DynBranch depends on neither the agent loop nor the protocol.

Serving integration. DynBranch hooks three lifecycle events on every normalized arrival: stage arrival, the opening of an unresolved branch, and the canonical request carrying its resolved selection. The hooks reuse the serving path’s existing queue, slot, and KV accounting, the same CSL manager, and the same terminal settlement path; canonical requests and speculative fills execute through the existing SGLang or vLLM executors [39, 116]. Disabling DynBranch bypasses the hooks and preserves the original semantics.

5 Evaluation

We evaluate end-to-end latency, completion, and tail (§5.2); how width, replicas, and depth trade GPU work for latency (§5.3); what reuse (C1), early fill (C2), and each admission gate contribute (§5.4); and how the system holds up under rising and bursty load (§5.5). Task accuracy is checked against the floor in every experiment.

5.1 Experimental Setup

Testbed. Our main testbed is a dedicated server with two AMD EPYC 9755 CPUs, 2.2 TiB of main memory, and four NVIDIA H200 NVL GPUs with 141 GiB of HBM each. Workers serve Qwen3-32B [103] unless stated otherwise, and a distilled Qwen3-0.6B drafts branch candidates. The commodity replication in Figure 8(c,d) uses four distributed worker nodes on vast.ai, each serving Qwen3-8B on one NVIDIA RTX 4090 with 24 GiB of device memory.

Workloads. We instantiate the regimes in §2.1 with:

- **Routing:** BFCL function-calling records [66], replayed with their handler choices and arguments.
- **ReAct:** multi-hop questions from MuSiQUE [90] over a local HotpotQA-derived retrieval corpus [106].
- **Sub-Agent:** MuSiQUE-derived research sub-topics; the reuse-only run (Table 1) uses the same entity distribution.
- **HCI:** menu-offer/user-pick pairs from Schema-Guided Dialogue [71], with paired, sampled think gaps between offer and selection [14, 22]; latency is measured on the user turn that follows the gap.

Baselines. We compare five systems spanning completed-result reuse [94, 121], workflow scheduling, and agent-level speculation. InstCache [121] pre-populates an exact request cache from predicted instructions. Helium⁺ [94]¹ combines cache-aware scheduling with exact operator-result reuse. Parrot [52] exposes workflow dataflow as semantic variables. DSP [28] tunes speculative-plan depth under a latency–cost objective. SPAgent [34] speculates the next search-agent action and retains it in an in-request buffer.

Metrics. The reference throughout is a *no-reuse floor*: the same stack with speculation and cross-request reuse disabled.

¹Helium⁺ runs Helium’s cache-aware scheduling policy on the concurrent dispatch path and adds exact per-operator result caching.

Open-loop experiments report client-observed mean per-request wall time at arrival rate λ (req/s) as the primary metric, with p99 characterizing the tail; serial comparisons report mean change against that floor. Correctness is task accuracy relative to it; the absolute level is workload-specific and not comparable across workloads. Error bars are ± 1 s.d. over seeds.

5.2 End-to-End Comparison

DynBranch achieves the lowest mean latency across all four workloads, both load levels, and both hardware configurations (Figure 8). On 32B/H200, it improves over each workload’s strongest external baseline by 14–32% at low load. At load, the reductions are 15–31% on *Routing*, *Sub-Agent*, and *HCI*, and 6.2% on *ReAct*. Across these cells, reductions against the no-reuse floor span 46–66%. Task accuracy tracks the floor in every experiment below: promotion is lossless by construction (§3.3).

Early fill under load. C2 retains substantial benefit under load on *Sub-Agent* and *HCI*, reducing latency by 30% and 35% over C1-only, respectively; *Routing* gains 8.6%. On open-vocabulary *ReAct*, the gain falls from 16.0% at low load to 0.4% at load, while 53–55% of issued fills fail to match the resolved branch and are discarded at both points. The similar mismatch rates accompany different latency gains: under load the binding constraint is how much execution a fill can still hide, not how often the branch is guessed right (§5.5).

Completion and tail latency. Every arm completes the offered load with zero runtime failures. Relative to the no-reuse floor on 32B/H200, DynBranch lowers p99 by 11–33% at load and 19–36% at low load on *Routing*, *ReAct*, and *Sub-Agent*; *HCI* matches the floor at both points.

Commodity replication. The benefit generalizes to the smaller-model, distributed deployment: with Qwen3-8B on four RTX 4090s, DynBranch lowers mean latency by 6.9–37.6% relative to each cell’s strongest external baseline and 29.6–54.6% relative to its no-reuse floor (Figure 8c,d). C2 improves over C1-only in every cell.

5.3 Speculation Cost and Sensitivity

Width, replicas, and depth each buy latency with GPU work, and each saturates; the benefit holds across model sizes and backbone families.

Width. Wider fanout keeps lowering latency, but a growing share of what it adds is thrown away. At four workers, increasing fanout K from 0 to 3 lowers mean latency from 18.2 to 15.2 s and adds 19.8 GPU-seconds of speculative work per request (Figure 9a). The reduction becomes statistically significant at $K=2$ (Welch’s t -test [98] against $K=0$). The discarded share rises from 20% at $K=1$ to 33% at $K=3$. Waste shifts from selected fills that fail to promote to fills for unselected candidates.

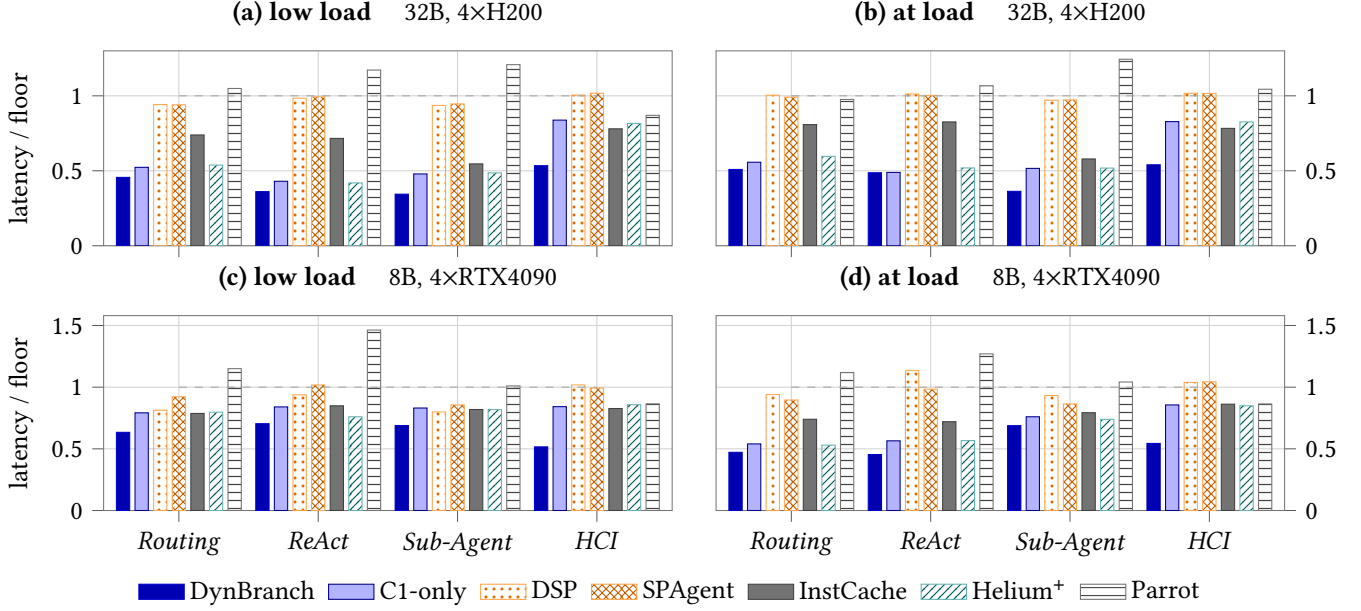


Figure 8. Open-loop mean latency, normalized to each workload’s own no-reuse floor (dashed = 1.0). Rows are hardware; columns are load, at $\lambda=0.12$ (*HCI* 0.3) and 0.8 req/s on H200 and at lower per-workload rates on RTX 4090.

Cancellation releases capacity promptly but cannot recover spent work: abort is confirmed 3 ms after branch resolution, by which point a cancelled fill has consumed 4.1 s of engine time against 6.2–6.5 s for a completed one.

Replicas. Replicas stop paying after the third worker. Scaling from one to four workers with $K=W-1$ at fixed per-GPU load reduces mean latency from 22.3 to 15.2 s (Figure 9b). The fourth worker saves only 0.25 s while adding 1.9 GPU-seconds of discarded work per request. Consumed work is nearly flat once width reaches the workload’s average of 2.67 candidates per branch. Task accuracy shows no regression across the seven width and replica configurations.

Depth. On *ReAct*, speculating beyond one run-ahead cycle (one hop ahead) does not pay. One cycle captures the useful

overlap in the hop chain, reducing median session wall time by 11.8% and serving 60 of 96 arrivals. Two or three cycles reach the additional hops but consume no results from them; latency changes by at most 0.7%, within the observed 2–5% run-to-run spread. Deeper speculative decisions finish too late to precede canonical work.

Repetition and model size. More recurrence increases relative gain; larger serving models increase absolute savings (Figure 10). Raising exact replay from 0.22 to 0.77 increases mean-latency speedup from 1.8× to 8.6×. Across Qwen3 0.6B–32B models, speedup stays at 1.75–2.06× while savings grow from 1.6 to 8.9 s per request.

Backbone family. Speedup varies little with the served family. Four alternative backbones (Figure 11a) yield 1.55–1.99× speedup over their paired no-reuse floors across three seeds each, with task accuracy within 0.012 of the floor.

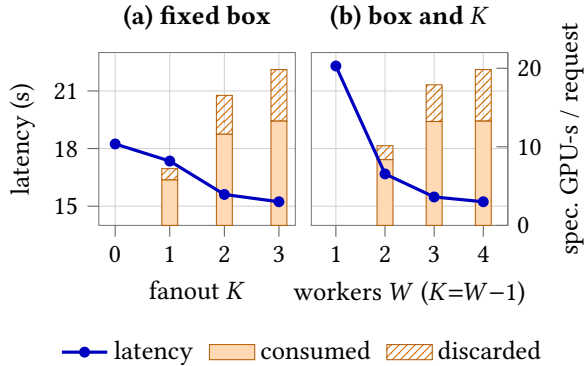


Figure 9. Speculation width and scale on *Routing*. (a) Fixed four-worker box, K swept. (b) Box and K grown together ($K=W-1$).

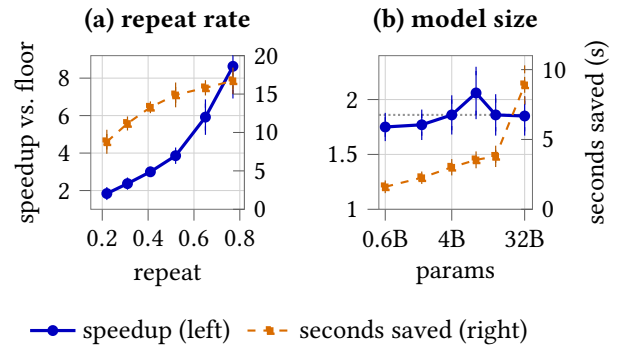


Figure 10. Sensitivity on *Routing* to (a) exact-replay rate and (b) serving-model size (dotted: mean over sizes).

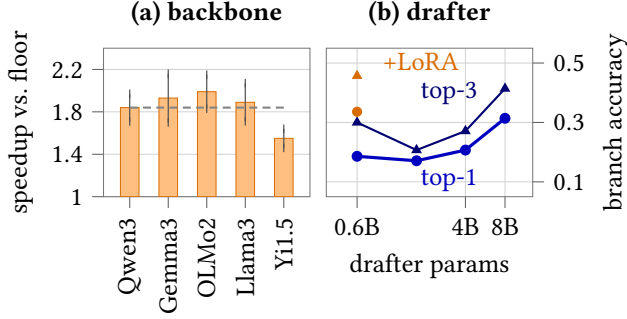


Figure 11. Sensitivity to the models. (a) Served backbone on *Routing* (dashed: Qwen3-32B). (b) Drafter size on *ReAct*.

Drafter variants. Higher branch-prediction accuracy does not by itself improve end-to-end latency: a prediction must also yield its result in time for demand. Across 0.6B–8B drafters, mean-latency speedup stays at 1.63–1.70 $\times$, with no statistically significant difference between sizes across three seeds; every size improves over the floor. The 8B drafter has the highest top-1 accuracy over 140 held-out branches (Figure 11b) and issues the most fills, but has the lowest consumption rate, 50% versus 70% for 1.7B.

The drafter need not match the served model’s family: Llama-3.2-1B matches Qwen3-1.7B in top-1 accuracy, and Qwen2.5-3B exceeds Qwen3-4B. Distilling 32B branch decisions into a 0.6B LoRA drafter yields 0.34 top-1 accuracy, exceeding the 8B drafter at a little over half its decode cost (+LoRA in Figure 11b).

5.4 Ablation Study

C1 captures reuse across partially overlapping requests; C2 lowers latency even with cross-request reuse disabled. Both preserve task accuracy against their paired no-reuse floors. A final pair of experiments ablates the two admission gates.

Reuse alone. With C2 disabled, DynBranch reduces mean latency on partial-overlap requests by 31%, matching Helium⁺’s 30% reduction; InstCache [121] gains only 7% (Table 1). The workload mixes exact repeats, shared sub-DAGs, and fresh negative controls; arms are paired by request id on cold-started stacks. Exact repeats reach the cache-hit path; fresh requests yield no reuse benefit. Across the mix, DynBranch reduces mean latency by 32.2%, with zero unsafe splices. C1 thus captures reuse below the whole-request boundary while retaining write-cascade invalidation.

Early fill alone. With C1 disabled, DynBranch reduces mean latency by 8.5–47.7% against the shared no-reuse floor across all four workloads on both two- and four-GPU deployments (Table 2). The baselines run their native admission, and DynBranch outperforms DSP [28] and SPAgent [34] throughput. These gains overlap with C1’s: some filled stages would also be served by reuse.

Arm	repeat	partial overlap	fresh
floor	14.4 s	14.8 s	13.6 s
InstCache	0.01 s	13.8 s (–7%)	14.6 s
Helium ⁺	0.16 s	10.3 s (–30%)	14.0 s
DynBranch	0.01 s	10.2 s (–31%)	14.1 s

Table 1. Reuse alone by request type (Qwen3-32B, paired by request id); *partial overlap* = a sub-DAG shared with another record.

System	<i>Routing</i>	<i>ReAct</i>	<i>Sub-Agent</i>	<i>HCI</i>
<i>2 GPUs</i>				
DynBranch	–18.6	–8.7	–34.4	–46.6
SPAgent	–12.3	–0.7	–23.6	–1.0
DSP	–13.1	–0.4	–23.8	–0.7
<i>4 GPUs</i>				
DynBranch	–21.9	–8.5	–34.0	–47.7
SPAgent	–10.6	+0.7	–25.0	+1.6
DSP	–11.2	–0.2	–24.7	–0.2

Table 2. C2 isolation (serial closed-loop): mean $\Delta\%$ against the shared no-reuse floor, C1 off on every arm.

The gain extends beyond a single predicted action. On *Routing*, 349 fills yield 500 consumed stages through chained promotion. *HCI* shows the largest separation: completed fills serve 73% of choices, cutting user-turn wait by 46.6–47.7%; neither baseline materially benefits from the think gap.

Case study. Early fill removes four downstream stages from the critical path in the paired *Routing* request in Figure 12, saving 5.75 s end to end. The request computes an area and then converts its units. Two fills overlap router decoding on other workers, each completing a handler and its executor before demand, so they are served at zero engine time. The floor instead runs all four after the decisions that gate them, for 6.50 s in total, close to the observed end-to-end saving.

Both arms take the same route, and this request has the median paired saving. Across the run, 99% of promotions find a completed fill and 1% wait for one. Producer identities distinguish speculative consumption from ordinary cache hits; attributed stage counts match the system’s own counter.

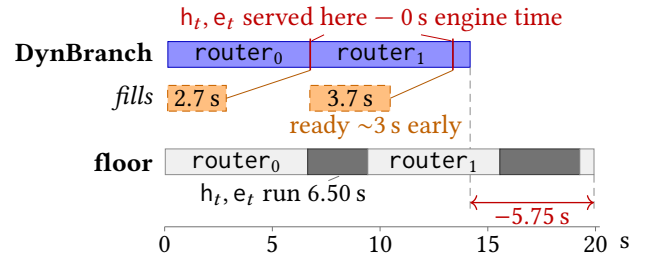


Figure 12. A paired *Routing* request. Each decision (router_t) gates a handler and executor (h_t, e_t).

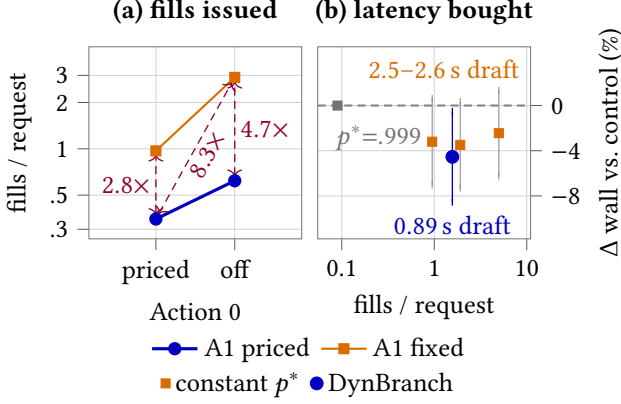


Figure 13. Two-level admission on *ReAct*. (a) The 2×2 ablation. (b) The same gate vs. fixed thresholds (95% CIs).

Two-level attribution. Pricing both levels issues $8.3\times$ fewer fills than disabling Action 0 and fixing Action 1, at indistinguishable latency (Figure 13a at $\lambda=0.80$; $n=155$, paired within batch with arm order reversed between batches). Each level controls a distinct cost: Action 0 cuts candidate-generation openings from 1.53 to 0.35 per request with Action 1 priced, and from 1.42 to 0.50 with it fixed; Action 1 reduces fill execution under either setting of Action 0.

Generation cost. Only Action 0 removes drafting cost; a fill threshold cannot. On *ReAct* with C1 disabled, we disable Action 0, so the drafter runs at every branch, and replace Action 1 with fixed thresholds p^* . Across these thresholds, fills per request vary $55\times$ (0.09–4.99), yet drafting stays at 2.5–2.6 s per request (Figure 13b). Against $p^*=0.999$, which issues almost no fills, the fixed thresholds lower latency by 2.5–3.5%, within noise (their CIs include zero); DynBranch with both gates priced lowers it by 4.5% (95% CI 0.5–8.5%, $n=174$ over three order-balanced seeds at $\lambda=0.12$) while drafting 0.89 s, $2.8\times$ less. Task accuracy spans 0.305–0.328 across the sweep.

5.5 Stress Test

Under rising and bursty load, priced admission keeps the benefit that fixed policies lose.

Load decomposition. C1’s saving follows the request mix rather than the rate; C2’s follows load. Across five open-loop *Routing* rates, C1 removes 40–48% of the floor latency; C2 removes a further 0.25–1.39 s (Figure 14). The floor, C1-only, and full arms replay the same schedule, giving Δ_{C1} = floor – C1only and Δ_{C2} = C1only – full; the three stack to the no-reuse floor in Figure 14a. All requests complete. C1-only and full have identical task accuracy.

C2’s gain falls by 82% from its peak as load rises but remains positive at every measured rate. Fills issued fall from 1.4 to 0.9 per request, while latency saved per fill falls by 72%; the latter accounts for about three quarters of the narrowing. The gain therefore depends on both admission volume and

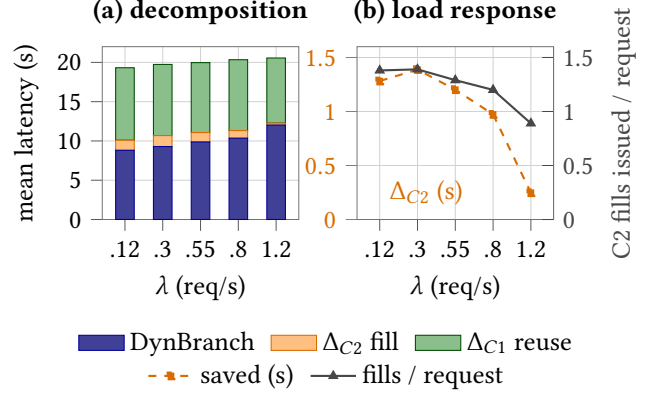


Figure 14. *Routing* under load, three seeds. (a) Latency decomposition. (b) What C2 saves and the fills it issues.

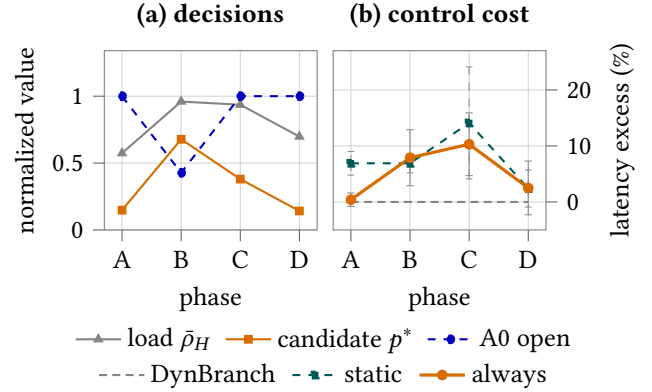


Figure 15. Phased adaptation on *Routing*. (a) What the two levels decide. (b) What the simple controls cost.

how much execution each fill hides within the remaining resolution window.

Dynamic adaptation. Priced admission lowers complete-trace mean latency to 8.09 s, 5.7% below **static** and 5.5% below **always** (Figure 15b). Static opens candidate generation only with at least three free workers and queue depth at most two; always opens whenever a candidate exists. Task accuracy is 0.977 for DynBranch and static and 0.979 for always, with no failures.

The trace brackets a burst (B) and a reuse-heavy phase (C) with low-load (A) and recovery (D) phases. B and C have the same arrival rate, but C1 absorbs more work in C. The share of candidate-opening branches rises from 43% in B to 100% in C, while the break-even threshold (candidate-set mean, phase median) falls from 0.678 to 0.380 (Figure 15a).

5.6 Limitations and Future Work

The evaluated workflows use homogeneous serving models, stable branch coordinates before resolution, and side-effect-free speculative stages. Benefit depends on recurrence, predictability, available overlap, and spare capacity. Extensions

include templates from emitted plans [112], transactional isolation and cross-resource scheduling for stateful tools and heterogeneous multimodal pipelines [17, 89], and speculative units ranging from retrieval or prefill [96, 114] to branches.

6 Related Work

LLM state and result caches. Whole responses are cached under exact or semantic query matches [9, 45, 121], plan templates are reused [112], prefix or chunk KV state is retained, blended, or routed [2, 16, 24, 64, 85, 107, 110, 115, 116], and tool outputs are memoized [10, 76, 92]. Continuum preserves KV state across tool-call pauses using tool-aware lifetimes and program-level scheduling [44]. These systems retain content or model state whose identity is already known. DynBranch instead produces work before the branch outcome exists, then promotes the exact selected result with its dependency provenance for later reuse.

Reusable computation and freshness. Semantic caching and answering queries using views established sub-query reuse [19, 30]; derived state can be maintained incrementally [11, 29], as Noria, ReadySet, and Differential Dataflow do in dataflow systems [27, 57, 72]. Freshness scheduling and self-adjusting computation track when derived work remains valid [1, 40]. Adaptive query processing and multi-query optimization revise and share execution plans [6, 35, 78]. DynBranch adopts the same principle (reusable work needs an explicit dependency and freshness contract) for an LLM subgraph whose dependencies are external reads, not base tables.

Agent-aware serving. Recent characterization finds heterogeneous resources, long idle intervals, shifting bottlenecks, and cross-request tool redundancy in agentic workloads [15]. Agentix treats programs as first-class scheduling units [55], while a policy-driven agent runtime exposes observation, prediction, scoring, and action hooks between frameworks and engines, evaluating agent-aware KV prefetch and retention [113]. Murakkab configures workflow components across models and hardware under SLOs [17], and Dyserve adapts per-node model, verifier, and backend choices under load [69].

Workflow and dataflow optimization. Helium couples proactive caching with workflow scheduling, while Halo plans model residency for known DAGs [79, 94]. FlowMesh deduplicates operator work by content, and Lumilake merges identical intermediate nodes across known jobs [80, 87]. Session DAG and stage-pipeline systems make their inter-stage dependencies explicit to the scheduler [38, 52, 89]. Agentix, Alto, and Conveyor add program-aware preemption, nested-ancestry dataflow orchestration, and streamed tool/decode overlap [55, 70, 102]. DynBranch supplies a dependency not yet visible to the scheduler and returns the confirmed result

to the same dataflow, so these techniques remain complementary.

Speculative agents and serving. Interactive Speculative Planning, dialogue-response prefetching, Speculative Actions, DSP, SpecHop, SPORK, and SPEX predict future actions, reasoning paths, or downstream work and verify the realized path [8, 28, 33, 59, 77, 109, 117]. Skim executes a site-template fast path and verifies its output before falling back to the full agent [100]; Sherlock instead overlaps a produced node’s verification with its downstream work [75]. Tool-oriented systems execute predicted calls [32, 88]; PASTE, SPAGent, IdleSpec, B-PASTE, and DualSpec gate them on confidence, utility, capacity, or semantic consistency [18, 34, 83, 88, 118]. PASTE is closest: beyond the hold-until-match of §2 it co-schedules tools with returning LLM sessions, but still consumes the confirmed result within its originating request. Speculate-with-Memory carries predictor statistics and episodes across tasks [50], rather than persisting validated products. Token-level speculative decoding drafts and verifies individual tokens [13, 43, 49, 119], and serving systems adapt that draft work to load, batch, or budget [47, 48, 51, 53, 54]. A separate family lowers latency by changing what the model computes: routing to a cheaper model [62, 69], or bounding the reasoning budget, which can change the answer itself [42, 60]. DynBranch overlaps candidate subgraphs with resolving decode; completed results serve later requests after exact matching and freshness checks. DynBranch reprices each branch at generation and every fill using live load and observed outcomes.

7 Conclusion

DynBranch removes the branch-resolution barrier by treating an unresolved branch as serving state: a coordinate names candidate work before the branch resolves, an exact input match with fresh dependencies authorizes it afterwards, and a two-level controller prices both against live load. Across four agentic workloads it cuts mean latency by up to 32% over the strongest prior system and by 46–66% against a no-reuse floor, with workflow results unchanged; with reuse disabled, early fill alone still removes 8.5–47.7%. The contract behind that result rests only on a named decision point and a stage safe to run twice, so it carries to workflows whose shape is decided at runtime, and priced admission sustains it from idle to saturated load. Within those two requirements, predictors and pricing policies can evolve without weakening exact reuse or freshness.

References

- [1] Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2002. Adaptive Functional Programming. In *POPL*.
- [2] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. *Proceedings of the ACM on*

- Management of Data (PACMOD)* 3, 3 (2025), 1–28. arXiv:2502.15734.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2403.02310.
 - [4] Jonatha Anselmi and Neil Walton. 2022. Stability and Optimization of Speculative Queueing Networks. *IEEE/ACM Transactions on Networking* 30, 2 (2022), 911–922.
 - [5] Anthropic. 2025. Claude Code Documentation. <https://code.claude.com/docs>. Accessed 2026-07.
 - [6] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*.
 - [7] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *arXiv preprint arXiv:2505.20411* (2025).
 - [8] Huajun Bai, Weiwei Lv, Huichuan Zheng, Youyou Lu, and Jiwu Shu. 2026. SPORK: Self-Speculative Forking to Accelerate Agentic LLM Inference. arXiv:2607.03333.
 - [9] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*.
 - [10] Farhana Begum, Craig Scott, Kofi Nyarko, Mansoureh Jeihani, and Fahmi Khalifa. 2026. Hierarchical Caching for Agentic Workflows: A Multi-Level Architecture to Reduce Tool-Execution Overhead. *Machine Learning and Knowledge Extraction (MDPI)* 8, 2 (2026), 30. doi:10.3390/make8020030
 - [11] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. 1986. Efficiently Updating Materialized Views. In *SIGMOD*.
 - [12] Jake Brutlag. 2009. Speed Matters for Google Web Search. Google Research.
 - [13] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *ICML*.
 - [14] Stuart K. Card, Thomas P. Moran, and Allen Newell. 1980. The Keystroke-Level Model for User Performance Time with Interactive Systems. *Commun. ACM* 23, 7 (1980), 396–410.
 - [15] Chaokun Chang, Yukun Zhou, Kaihua Fu, Dakai An, Tianyu Feng, Hanfeng Lu, Sheng Yao, Pu Guo, Yinghao Yu, Yizhou Shan, Bo Li, Binhang Yuan, and Wei Wang. 2026. From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems. arXiv:2608.15127.
 - [16] Character.AI. 2024. Optimizing AI Inference at Character.AI. Character.AI Research Blog. Archived at <https://web.archive.org/web/20250907181248/https://research.character.ai/optimizing-inference/>. Original URL <https://research.character.ai/optimizing-inference/> now redirects to the blog index; accessed 2026-09.
 - [17] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. 2026. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 567–587.
 - [18] Daewon Choi, Kyunghyun Park, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Jinwoo Shin, and Aram Galstyan. 2026. IdleSpec: Exploiting Idle Time via Speculative Planning for LLM Agents. *arXiv preprint arXiv:2605.22154* (2026).
 - [19] Shaul Dar, Michael J. Franklin, Björn Thór Jónsson, Divesh Srivastava, and Michael Tan. 1996. Semantic Data Caching and Replacement. In *VLDB*.
 - [20] Brian D. Davison and Haym Hirsh. 1998. Predicting Sequences of User Actions. In *AAAI/ICML Workshop on Predicting the Future: AI Approaches to Time-Series Analysis*.
 - [21] DeepSeek-AI. 2026. DeepSeek Harness: Everything is a Plugin. <https://github.com/deepseek-ai/deepseek-harness>. Documentation at <https://deepseek-harness.github.io/deepseek-harness/>. Accessed 2026-09.
 - [22] W. J. Doherty and A. J. Thadhani. 1982. *The Economic Value of Rapid Response Time*. Technical Report GE20-0752-0. IBM.
 - [23] Stephen Fitchett and Andy Cockburn. 2012. AccessRank: Predicting What Users Will Do Next. In *ACM CHI Conference on Human Factors in Computing Systems*.
 - [24] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cached Attention. In *USENIX Annual Technical Conference (ATC 24)*.
 - [25] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv:2402.14034.
 - [26] Wei Gao, Yuheng Zhao, Dilxat Muhtar, Dakai An, Xuchun Shang, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Weixun Wang, Ju Huang, Teng Ma, Siran Yang, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2026. ROSE: Rollout On Serving GPUs via Cooperative Elasticity for Agentic RL. arXiv:2605.06534.
 - [27] Jon Gjengset, Malte Schwarzkopf, Jonathan Behrens, Lara Timbó Araújo, Martin Ek, Eddie Kohler, M. Frans Kaashoek, and Robert Morris. 2018. Noria: Dynamic, Partially-Stateful Data-Flow for High-Performance Web Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*. USENIX Association, Carlsbad, CA, 213–231. <https://www.usenix.org/conference/osdi18/presentation/gjengset>
 - [28] Yilin Guan, Qingfeng Lan, Fei Sun, Dujian Ding, Devang Acharya, Chi Wang, William Yang Wang, and Wenyue Hua. 2025. Dynamic Speculative Agent Planning. arXiv:2509.01920.
 - [29] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. 1993. Maintaining Views Incrementally. In *SIGMOD*.
 - [30] Alon Y. Halevy. 2001. Answering Queries Using Views: A Survey. *The VLDB Journal* 10, 4 (2001).
 - [31] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism. doi:10.64434/tml.20250910 <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
 - [32] Coleman Hooper, Minwoo Kang, Suhong Moon, Nicholas Lee, Eric Wen, John Wawrzyniek, Michael W. Mahoney, Yakun Sophia Shao, Amir Gholami, and Kurt Keutzer. 2026. Speculative Interaction Agents: Building Real-Time Agents with Asynchronous I/O and Speculative Tool Calling. arXiv:2605.13360.
 - [33] Wenyue Hua, Mengting Wan, Shashank Vadrevu, Ryan Nadel, Yongfeng Zhang, and Chi Wang. 2024. Interactive Speculative Planning: Enhance Agent Efficiency through Co-design of System and User Interface. arXiv:2410.00079.
 - [34] Zixiao Huang, Wen Zeng, Tianyu Fu, Tengxuan Liu, Yizhou Sun, Ke Hong, Xinhao Yang, Chengchun Liu, Yan Li, Quanlu Zhang, Guohao Dai, Zhenhua Zhu, and Yu Wang. 2025. Reducing Latency of LLM Search Agent via Speculation-based Algorithm-System Co-Design. *arXiv preprint arXiv:2511.20048* (2025).
 - [35] Navin Kabra and David J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *SIGMOD*.
 - [36] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense

- Passage Retrieval for Open-Domain Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2004.04906.
- [37] Emilie Kaufmann, Olivier Cappé, and Aurélien Garivier. 2012. On Bayesian Upper Confidence Bounds for Bandit Problems. In *Proceedings of the 15th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 592–600.
- [38] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *ICML*.
- [39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.
- [40] Alexandros Labrinidis and Nick Roussopoulos. 2003. Balancing Performance and Data Freshness in Web Database Servers. In *VLDB*.
- [41] LangChain. 2025. LangGraph: Low-Level Orchestration Framework for Building Stateful Agents. <https://docs.langchain.com/oss/pytho n/langgraph/overview>. Accessed 2026-09.
- [42] Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, et al. 2023. Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702.
- [43] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *ICML*.
- [44] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph E. Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230.
- [45] Jiaxing Li, Chi Xu, Feng Wang, Isaac von Riedemann, Cong Zhang, and Jiangchuan Liu. 2024. SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models. In *IEEE/ACM International Symposium on Quality of Service (IWQoS)*.
- [46] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. 661–670. doi:10.1145/1772690.1772758
- [47] Ruixiao Li, Fahao Chen, and Peng Li. 2025. Semi-Clairvoyant Scheduling of Speculative Decoding Requests to Minimize LLM Inference Latency. In *IJCAI*.
- [48] Rui Li, Zhaoning Zhang, Libo Zhang, Huaimin Wang, Xiang Fu, and Zhiquan Lai. 2025. Nightjar: Dynamic Adaptive Speculative Decoding for Large Language Models Serving. *arXiv preprint arXiv:2512.22420* (2025).
- [49] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EA-GL: Speculative Sampling Requires Rethinking Feature Uncertainty. In *ICML*.
- [50] Yu Li, Qinyuan Ye, Prafulla Kumar Choubey, Jiaxin Zhang, and Chien-Sheng Wu. 2026. Speculate with Memory: Lossless Acceleration for LLM Agents. arXiv:2607.12236.
- [51] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Yi-Hsiang Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. 2026. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. In *EuroSys*.
- [52] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *OSDI*. arXiv:2405.19888.
- [53] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. Online Speculative Decoding. In *ICML*.
- [54] Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Chen Zhang, Kuntai Du, Xiangxi Mo, Kaichao You, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. TurboSpec: Closed-loop Speculation Control System for Optimizing LLM Serving Goodput. *arXiv preprint arXiv:2406.14066* (2024).
- [55] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2026. Agentix: An Efficient Serving Engine for LLM Agents as General Programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, Renton, WA, 2443–2459. <https://www.usenix.org/conference/nsdi26/presentation/luo> Earlier preprint titled Autellix, arXiv:2502.13965.
- [56] Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. 2025. Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories. arXiv:2511.00197.
- [57] Frank McSherry, Derek G. Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential Dataflow. In *CIDR*.
- [58] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunhan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *ASPLOS*. doi:10.1145/3620666.3651335
- [59] Kiyotada Mori, Seiya Kawano, Angel Fernando Garcia Contreras, and Koichiro Yoshino. 2025. Dialogue Response Prefetching Based on Semantic Similarity and Prediction Confidence of Language Model. In *INTERSPEECH*.
- [60] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple Test-Time Scaling. arXiv:2501.19393.
- [61] Robert O’Callahan, Chris Jones, Nathan Froyd, Kyle Huey, Albert Noll, and Nimrod Partush. 2017. Engineering Record and Replay for Deployability. In *USENIX Annual Technical Conference (ATC)*.
- [62] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665.
- [63] OpenAI. 2025. Codex CLI Documentation. <https://developers.openai.com/codex/cli>. Accessed 2026-07.
- [64] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *arXiv preprint arXiv:2507.07400* (2025).
- [65] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*. arXiv:2311.18677.
- [66] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 48371–48392.
- [67] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *NeurIPS*.
- [68] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. 1995. Informed Prefetching and Caching. In *SOSP*.
- [69] Jiayi Qian, Yichong Zhang, Hanchen Yang, Chun Tao, Souvik Kundu, Zishen Wan, and Tushar Krishna. 2026. Serving Agentic Workflows with a Physical-Plan Compiler and Adaptive Runtime. arXiv:2607.02942.

- [70] Deepti Raghavan, Keshav Santhanam, Muhammad Shahir Rahman, Nayani Modugula, Luis Gaspar Schroeder, Maximilien Cura, Houjun Liu, Pratiksha Thaker, Philip Levis, and Matei Zaharia. 2024. Alto: Orchestrating Distributed Compound AI Systems with Nested Ancestry. *arXiv:2403.04311v3*.
- [71] Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. 2020. Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [72] Readysset. 2022. Introducing Readysset: High-Performance SQL Caching Engine. <https://readysset.io/blog/introducing-readysset>.
- [73] Pol G. Recasens, Ferran Agullo, Yue Zhu, Chen Wang, Eun Kyung Lee, Olivier Tardieu, Jordi Torres, and Josep Ll. Berral. 2025. Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference. In *IEEE International Conference on Cloud Computing (CLOUD)*. *arXiv:2503.08311*.
- [74] Sajal Regmi and Chetan Phakami Pun. 2024. GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching. *arXiv preprint arXiv:2411.05276* (2024).
- [75] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. 2025. Sherlock: Reliable and Efficient Agentic Workflow Execution. *arXiv:2511.00330*.
- [76] Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. 2026. Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, Renton, WA, 2407–2421. <https://www.usenix.org/conference/nsdi26/presentation/ruan-cortex>
- [77] Mehrdad Saberi, Keivan Rezaei, and Soheil Feizi. 2026. SpecHop: Continuous Speculation for Accelerating Multi-Hop Retrieval Agents. *arXiv preprint arXiv:2605.21965* (2026).
- [78] Timos K. Sellis. 1988. Multiple-Query Optimization. *ACM TODS* 13, 1 (1988), 23–52. doi:10.1145/42201.42203
- [79] Junyi Shen, Noppanat Wadlom, and Yao Lu. 2025. Batch Query Processing and Optimization for Agentic Workflows. *arXiv preprint arXiv:2509.02121* (2025).
- [80] Junyi Shen, Noppanat Wadlom, Lingfeng Zhou, Dequan Wang, Xu Miao, Lei Fang, and Yao Lu. 2025. FlowMesh: A Service Fabric for Composable LLM Workflows. *arXiv preprint arXiv:2510.26913* (2025).
- [81] Zhengliang Shi, Yuhang Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. 2025. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. In *Findings of the Association for Computational Linguistics (ACL Findings)*. *arXiv:2503.01763*.
- [82] James E. Smith. 1981. A Study of Branch Prediction Strategies. In *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*. 135–148.
- [83] Yanfei Song. 2026. B-PASTE: Beam-Aware Pattern-Guided Speculative Execution for Resource-Constrained LLM Agents. *arXiv:2604.16469*
- [84] Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. 2007. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. In *HPCA*.
- [85] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. 2025. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. In *International Conference on Learning Representations (ICLR)*.
- [86] Qidong Su, Christina Giannoula, and Gennady Pekhimenko. 2023. The Synergy of Speculative Decoding and Batching in Serving Large Language Models. *arXiv preprint arXiv:2310.18813* (2023).
- [87] Zhengyuan Su, Noppanat Wadlom, Junyi Shen, Yicong Huang, Wentao Wu, and Yao Lu. 2026. Lumilake: An Agentic Analytics Engine for AI4Science. Proceedings of the Supporting Our AI Overlords workshop at the ACM Conference on AI and Agentic Systems (SAO@CAIS 2026). <https://www.microsoft.com/en-us/research/publication/lumilake-an-agentic-analytics-engine-for-ai4science/>
- [88] Yifan Sui, Han Zhao, Rui Ma, Zhiyuan He, Hao Wang, Jianxun Li, Kaiqiang Xu, Kai Chen, and Yuqing Yang. 2026. Parallelizing Tool Execution and LLM Generation for Low-Latency Agent Serving. *arXiv:2603.18897v3* (rev. 2026-06-16).
- [89] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*. Association for Computing Machinery, Rotterdam, Netherlands, 1302–1316. doi:10.1145/3676641.3716278 Earlier preprint titled Teola, *arXiv:2407.00326*.
- [90] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics (TACL)* 10 (2022).
- [91] Claire Vernade, Alexandra Carpentier, Tor Lattimore, Giovanni Zappella, Beyza Ermis, and Michael Brückner. 2020. Linear Bandits with Stochastic Delayed Feedback. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 119)*. 9712–9721. *arXiv:1807.02089*.
- [92] Abhishek Vijaya Kumar, Bhaskar Kataria, Byungsoo Oh, Emaad Manzoor, and Rachee Singh. 2026. TVCache: A Stateful Tool-Value Cache for Post-Training LLM Agents. *arXiv:2602.10986*.
- [93] Ashish Vulimiri, P. Brighten Godfrey, Radhika Mittal, Justine Sherry, Sylvia Ratnasamy, and Scott Shenker. 2013. Low Latency via Redundancy. In *ACM CoNEXT*.
- [94] Noppanat Wadlom, Junyi Shen, and Yao Lu. 2026. Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. *arXiv preprint arXiv:2603.16104* (2026).
- [95] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KV-Cache Cache at a Large Cloud Provider. In *USENIX Annual Technical Conference (ATC)*. *arXiv:2506.02634*.
- [96] Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2025. Speculative RAG: Enhancing Retrieval Augmented Generation through Drafting. In *ICLR*.
- [97] Martin L. Weitzman. 1979. Optimal Search for the Best Alternative. *Econometrica* 47, 3 (1979), 641–654.
- [98] B. L. Welch. 1947. The Generalization of 'Student's' Problem when Several Different Population Variances Are Involved. *Biometrika* 34, 1-2 (1947), 28–35. doi:10.1093/biomet/34.1-2.28
- [99] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [100] Mike Wong, Kevin Hsieh, Suman Nath, and Ravi Netravali. 2026. Skim: Speculative Execution for Fast and Efficient Web Agents. *arXiv:2605.16565*.
- [101] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*.
- [102] Yechen Xu, Xinhao Kong, Tingjun Chen, and Danyang Zhuo. 2024. Conveyor: Efficient Tool-aware LLM Serving with Tool Partial Execution. *arXiv:2406.00059*.

- [103] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388.
- [104] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [105] Sen Yang, Shujian Huang, Xinyu Dai, and Jiajun Chen. 2024. Multi-Candidate Speculative Decoding. *arXiv preprint arXiv:2401.06706* (2024).
- [106] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.
- [107] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *European Conference on Computer Systems (EuroSys)*. arXiv:2405.16444.
- [108] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.
- [109] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kafkes, and Tianyi Peng. 2025. Speculative Actions: A Lossless Framework for Faster Agentic Systems. *arXiv preprint arXiv:2510.04371* (2025).
- [110] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. *arXiv preprint arXiv:2604.25899* (2026).
- [111] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, Dunjie Lu, Haikong Lu, Qi Zhen, Xinyuan Wang, Jiaqi Deng, Yuhao Yang, Cheng Chen, Boyuan Zheng, Alex Su, Xiao Yu, Hao Zou, Saaket Agashe, Xing Han Lu, Manpreet Kaur, Zhengyang Qi, Vincent Sunn Chen, Frederic Sala, Dayiheng Liu, Junyang Lin, Zhou Yu, Yu Su, Siva Reddy, Xin Eric Wang, Peng Qi, Tianbao Xie, and Tao Yu. 2026. OSWorld 2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. arXiv:2606.29537.
- [112] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *NeurIPS*. doi:10.52202/085713-3451 arXiv:2506.14852.
- [113] Rui Zhang, Chaeun Kim, and Liting Hu. 2026. A Policy-Driven Runtime Layer for Agentic LLM Serving. arXiv:2605.27744.
- [114] Wuyang Zhang and Shichao Pei. 2026. Predictive Prefetching for Retrieval-Augmented Generation. In *ICML*.
- [115] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. *arXiv preprint arXiv:2605.06472* (2026).
- [116] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*.
- [117] Shuzhang Zhong, Haochen Huang, Shengxuan Qiu, Pengfei Zuo, Runsheng Wang, and Meng Li. 2026. Breaking the Reward Barrier: Accelerating Tree-of-Thought Reasoning via Speculative Exploration. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. arXiv:2605.10195.
- [118] Shuzhang Zhong, Baotong Lu, Qi Chen, Chuanjie Liu, Fan Yang, and Meng Li. 2026. DualSpec: Accelerating Deep Research Agents via Dual-Process Action Speculation. arXiv:2603.07416.
- [119] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. DistillSpec: Improving Speculative Decoding via Knowledge Distillation. In *ICLR*.
- [120] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560.
- [121] Longwei Zou, Yan Liu, Jiamu Kang, Tingfeng Liu, Jiangang Kong, and Yangdong Deng. 2024. InstCache: A Predictive Cache for LLM Serving. arXiv:2411.13820.

A Guarantees and Complexity of Reuse and Admission

This appendix derives the guarantees behind DynBranch. Two results concern C1 and C2: the subgraph key is the coarsest reuse key that is safe without a model of the stage, so it maximizes the reuse of Equation (1), and promotion is correspondingly the most permissive safe rule. For C3, we analyze Action 0’s branch-level sign decision, then establish exact per-candidate break-even, stepwise optimality, and terminal maximality of Action 1’s sequential repricing. An exact ordered-subset oracle checks the latter.

Setting. Fix a realized execution domain $\mathcal{E}$. An execution $e \in \mathcal{E}$ of a speculable stage $v(e)$ renders an input $q(e)$, reads external records at versions $D(e)$, and has semantic input

$$t(e) = (\text{fp}(v(e)), q(e), D(e)),$$

where the stage fingerprint covers its code, model, and execution parameters. It produces $y(e) = F(t(e))$, where F is deterministic: equal inputs and dependency versions give equal outputs (§3.3). Write $x(e) = (t(e), z(e))$ for the metadata available before the stage executes, where z is any further recorded field, such as unrelated parts of the request. A keying function κ is *input-based* if $\kappa(x)$ is a fixed function of that metadata for every possible F : it neither executes nor models F , and cannot inspect the stage’s outputs or intermediate results to decide a match. Every cache that does not model the stage it serves is input-based in this sense.

Let $\mathcal{F}$ be the family of stage semantics the serving layer admits—every deterministic function of t , since nothing further about F is known to it. Call κ *uniformly sound* if for every $F \in \mathcal{F}$ and all $e_1, e_2 \in \mathcal{E}$, $\kappa(x_1) = \kappa(x_2)$ implies $y(e_1) = y(e_2)$. Write

$$\kappa^*(x) = (\text{fp}(v), h(q), D)$$

for DynBranch’s reuse name under its freshness guard (§3.2): the stage fingerprint, the rendered input under a hash h assumed collision-free on inputs in $\mathcal{E}$, and the dependency versions the guard certifies.

Lemma A.1 (Soundness). κ^* is uniformly sound.

Proof. $\kappa^*(x_1) = \kappa^*(x_2)$ gives $t(e_1) = t(e_2)$ because h is collision-free on the realized inputs. Determinism of F yields $y(e_1) = y(e_2)$. $\square$

Lemma A.2 (Coarsest sound key). Let κ be input-based and uniformly sound. Then $\kappa(x_1) = \kappa(x_2)$ implies $\kappa^*(x_1) = \kappa^*(x_2)$: the partition κ induces refines the one κ^* induces.

Proof. By contraposition. Suppose $\kappa^*(x_1) \neq \kappa^*(x_2)$. Then $t(e_1) \neq t(e_2)$. Because $\mathcal{F}$ contains every deterministic function of t , it contains some F with $F(t(e_1)) \neq F(t(e_2))$. Uniform soundness of κ on that F forces $\kappa(x_1) \neq \kappa(x_2)$; because an input-based κ is fixed independently of F , it cannot merge the pair for any other semantics either. $\square$

Corollary A.3 (Maximal hit rate). Over the same resident entries and subgraph at each decision point j , κ^* maximizes every P_j^{hit} in Equation (1) among uniformly sound input-based keys, and hence maximizes the expected saving $\mathbb{E}[\Delta T_{\text{reuse}}]$.

Proof. A coarser partition serves weakly more demands, and by Lemma A.2 no uniformly sound input-based key is coarser than κ^* . Soundness is Lemma A.1. Each T_j^{sub} is fixed, so maximizing every P_j^{hit} maximizes the sum. $\square$

A whole-response key, which requires the entire request to match, and a root-anchored prefix key, which requires every token up to v to match, both separate executions that κ^* merges whenever the difference lies outside v 's rendered input. Each is therefore a strict refinement on some workflow; Figure 7 exhibits one, where they serve 0 and 2 of the six taken nodes against κ^* 's 4. Corollary A.3 says that 4 is the top of that ladder and not merely its highest built rung. *Input-based* is necessary rather than cosmetic: a key that modeled F could merge distinct inputs with equal outputs, but that lies outside the exact input-identity contract of §3.2.

Corollary A.4 (Completeness of promotion). Fix the shadows eligible under C2's lifecycle and C1's freshness and provenance guards. Promotion—match on κ^* with a fresh recorded read-set—is the most permissive sound rule over that set: no demand-shadow pair that some uniformly sound input-based rule could authorize is refused.

Proof. A uniformly sound input-based rule admitting a pair that κ^* separates would contradict Lemma A.2. Every remaining pair has equal keys and is authorized. $\square$

With the soundness argument of §3.3, which shows promotion never serves a value canonical execution would not have produced, Corollary A.4 brackets C2 from the other side.

Proposition A.5 (Minimal graph-based write cascade). Assume complete read sets and producer edges, and consider invalidation rules using only this dependency graph, without

modeling stage semantics. For a write set W , let A contain its direct readers and every descendant that loses a valid producer. The reverse-index cascade invalidates exactly A . If E_A are the traversed producer edges and R_A the dependency references removed, it takes $O(|W| + |A| + |E_A| + |R_A|)$ time and $O(|A|)$ auxiliary space. No uniformly sound rule over this graph can invalidate a strict subset of A .

Proof. The reverse index finds exactly the direct readers, and graph traversal visits each affected node and edge once; removing a node retires each of its recorded dependencies once. Every node in A lies on a dependency path from a changed record. For each such node, the admitted semantic family contains a deterministic function in which that change reaches its output. A graph-only rule cannot establish that the output is unchanged, so retaining the node is not uniformly sound. A node outside A either has no such path or retains a valid producer, so the cascade need not invalidate it. With incomplete provenance, DynBranch may extend the cascade conservatively; soundness remains, but minimality is not claimed. $\square$

Lemma A.6 (Tight overlap bound). Fix the realized durations at branch k . Early execution removes at most $\min(T_k^{\text{gen}}, T_k^{\text{exec}})$ when the selected candidate is covered and zero otherwise. Expectation over coverage and summation give Equation (2). Equality is attainable when an eligible fill starts with the resolution window, runs without contention, and is consumed immediately after resolution.

Proof. Execution cannot overlap more than either the window or its own duration. The stated schedule attains their minimum; multiplying by the coverage indicator and taking expectations gives the bound. $\square$

Reuse-path complexity. For rendered-input size $|q|$, returned payload size $|y|$, and a read set of size r , content-key construction costs $O(|q|)$, an indexed probe is expected $O(1)$, deep-copying a hit costs $O(|y|)$, and constructing the sorted read-set version costs $O(r \log r)$. For K_b shadows and R_b dependency references on branch b , its index makes branch resolution and promotion $O(K_b + R_b)$ rather than linear in all shadows. Exceeding the CSL's capacity can add an eviction scan over the cache. Aside from payloads, N entries with R dependency references use $O(N + R)$ metadata; caps and TTLs bound this state.

The generation gate (Action 0). Action 0 learns one net return per branch (Equation (4)), charging setup and enabled fills once and crediting consumption only after a C1 miss. An unproductive expansion retains its setup charge; mutually exclusive candidates do not receive separate optimism bonuses. Write $m_c(x) = x^\top A_c^{-1} q_c$ and $r_c(x) = \hat{\sigma}_c \sqrt{x^\top A_c^{-1} x} \geq 0$. After cold trials, unaudited admission opens iff $m_c(x) + r_c(x) > 0$.

Proposition A.7 (Conditional Action 0 sign guarantee). Let $\mu_c(x) = \mathbb{E}[\hat{y}_b \mid c, x]$. At a trained, unaudited decision, $\mu_c(x) \leq$

$m_c(x) + r_c(x)$ makes every refusal satisfy $\mu_c(x) \leq 0$. If also $\mu_c(x) \geq m_c(x) - r_c(x)$, then $m_c(x) - r_c(x) > 0$ certifies positive conditional net return.

Proof. Refusal means $m_c(x) + r_c(x) \leq 0$; apply the assumed upper bound. The lower bound gives the second claim. $\square$

The radius scales with observed target RMS, not a calibrated confidence bound; Proposition A.7 is conditional. Cold trials and seeded audits gather feedback outside this sign decision. Pending trials count toward the cold budget and register a branch token before expansion; delayed feedback revises that trial’s target and q_c without adding another observation to A_c .

The fill gate (Action 1). Consider one exclusive branch with candidates $1, \dots, n$ and the estimates of §3.4: $\tilde{p}_s = \tilde{\pi}_s^{\text{sel}} \tilde{\pi}_s^{\text{use}} \in [0, 1]$ for useful consumption, $\hat{G}_s^\Delta \geq 0$ for the execution its consumption saves after C1 lookup, and $C_s = C_s(V_H) \geq 0$ for the delay its execution imposes when it is *not* consumed. At projected state V_H , write $U_s(V_H) = \tilde{p}_s \hat{G}_s^\Delta - (1 - \tilde{p}_s) C_s(V_H)$.

Proposition A.8 (Stepwise optimality and maximality). *At any fixed V_H with $\hat{G}_s^\Delta + C_s(V_H) > 0$, $U_s(V_H) > 0$ exactly when $\tilde{p}_s > C_s(V_H) / (\hat{G}_s^\Delta + C_s(V_H))$, the threshold p^* of Equation (7). If both terms vanish, $U_s = 0$ and the strict rule rejects the candidate; the break-even ratio 0/0 is undefined. Excluding seeded audits, each Action 1 admission maximizes immediate utility among the candidates that pass hard feasibility and atomic reservation at the current projected state. Every admitted fill is therefore value-positive at reservation. If Action 1 stops because the maximum utility is non-positive, no remaining candidate can acquire positive utility after any further admissions.*

Proof. When $\hat{G}_s^\Delta + C_s(V_H) > 0$, rearranging $U_s(V_H) > 0$ gives the threshold; equality gives $U_s(V_H) = 0$. In particular, $C_s = 0 < \hat{G}_s^\Delta$ gives threshold 0, and $\hat{G}_s^\Delta = 0 < C_s$ gives threshold 1. When both vanish, $U_s = 0$ directly. The algorithm examines candidates in decreasing current utility, discards reservation failures, and admits the first feasible positive one. After an admission, PROJECTLOAD only increases slot and KV occupancy. The price curves are componentwise monotone, while $\tilde{p}_s$, $\hat{G}_s^\Delta$, and $\hat{R}_s$ remain fixed within the decision; hence every remaining C_s can only rise and every U_s can only fall. The stopping condition is therefore preserved in all states reachable by further admissions. $\square$

The proposition characterizes the implemented dynamic process: its next choice maximizes current feasible utility, and its price-based terminal set is maximal—no positive-utility fill can be appended. These per-step properties do not imply maximum total utility over all feasible orders; Appendix B compares that total with an exact oracle.

Admission complexity. For K candidates and context dimension d , Action 0 observation costs $O(d^2)$; revising a

delayed target costs $O(d)$ without changing A_c or the sample count. The direct matrix inversion used to score a context costs $O(d^3)$ and stores $O(d^2)$ per branch class ($d = 6$ here). Action 1 rescans the remaining candidates after every projected reservation, taking $O(K^2)$ time and $O(K)$ space.

Appendix B uses exact dynamic programming over all ordered subsets in 756 archived snapshots. Among the 639 with positive optimum, the implemented rule selects an exact-oracle set in 626 (98.0%) and retains 99.36% of aggregate sequence-aware oracle utility. With a binding footprint budget, global sequence optimization contains 0/1 knapsack even at zero price: for K mutually exclusive candidates, set $\tilde{p}_s = 1/K$, $\hat{G}_s^\Delta = K v_s$, and KV footprint w_s for each item, with budget B and all other limits slack. Each feasible subset then has value $\sum_s v_s$ and weight $\sum_s w_s \leq B$, so the problem is NP-hard. For this projected-state update, exact ordered-subset dynamic programming takes $O(K2^K)$ time and $O(2^K)$ space.

Scope. The Action 1 guarantees apply to estimated utility over revealed candidates. Action 0’s sign guarantee concerns its aggregate branch return under the stated bound; it makes no claim about unrevealed candidates. The gates’ end-to-end latency effect is evaluated in §5.4–§5.5.

B Checks from Archived Runs

We use archived artifacts—recorded outputs and admission snapshots, no new serving runs—to check deterministic execution and to replay the implemented sequential repricing rule against an exact ordered-subset oracle. The replay script, input hashes, and per-decision results will be released with the artifact. The replay holds Action 0 fixed: closed branches reveal no candidate set for its exact oracle; Figure 13 reports the two-level ablation.

Deterministic execution. A three-seed *ReAct* reproducibility run enables deterministic inference. In its no-reuse floor, 38 repeated-question groups contain 96 executions and 87 within-group pairs. Every pair has identical final answer text, and every group has an identical hop count. Across the full and C1-only arms, hop counts also agree on all 116 paired question IDs. All nine request artifacts match their manifest hashes, which supports Lemma A.1’s determinism premise in this configuration.

Sequential-admission replay. We use all 756 admission snapshots from a three-seed *Routing* run, including nine startup warmup events. The remaining 747 events join to the request artifacts by workflow ID. Each snapshot contains two or three candidates, totaling 2,133, with the probability, gain, cost, footprint, and load values used by the gate. The applicable price functions and curves match the archived source; reconstructed costs agree with all logged costs within their printing precision.

For each snapshot, an exact dynamic program optimizes over all ordered subsets under recorded KV headroom. Projected state depends only on the selected subset’s cardinality and total footprint, so one best prefix per subset suffices. Both oracle and implementation use the archived estimates and production price curves, project slot and KV occupancy after each admission, and score $(s_1, \dots, s_m)$ by $J_{\text{seq}} = \sum_i U_{s_i}(V_{i-1})$. Seeded audits are disabled.

Measure	Repriced / exact oracle
Oracle-set match	626/639 (98.0%)
$\sum J_{\text{seq}} / \sum J_{\text{seq}}^*$	99.36%
Median $J_{\text{seq}} / J_{\text{seq}}^*$	99.86%
5th percentile	97.35%
Minimum	94.32%

Table B.1. Exact replay of the sequence-aware admission objective. All rows use the 639 snapshots with positive optimum; the other 117 have zero optimum and both methods select nothing.

Among the 639 positive-optimum snapshots, the implemented order attains the exact sequence value in 281. Another 345 select an oracle-optimal set in a lower-value order; only 13 differ in selected set. The rule retains 99.36% of aggregate oracle utility (Table B.1). All candidates together use at most 16.8% of recorded KV headroom, so the footprint-cap knapsack case does not arise in this archive. This replay gives an exact comparison for the logged estimates and revealed candidates; \$5.4–\$5.5 measure the end-to-end latency effect.